

Dato A vettore di lung. n e dato un indice $i \in [1..n]$ determinare l'elemento che finirebbe in posizione i se ordinassi A
In $O(n)$ Selection

Casi particolari

$i=1 \leadsto$ Trova il min $\leadsto$ ok $O(n)$

$i=n \leadsto$ max $''$

$i=5 \leadsto$ Trova il min per 5 volte $\leadsto O(n)$

$i = \frac{n}{1000} \leadsto$ Trova il min per $\frac{n}{1000}$ volte $\leadsto O(n^2)$

$\leadsto$ Heapsort e cest. $A[\frac{n}{1000}] \leadsto O(n \log n)$



In generale

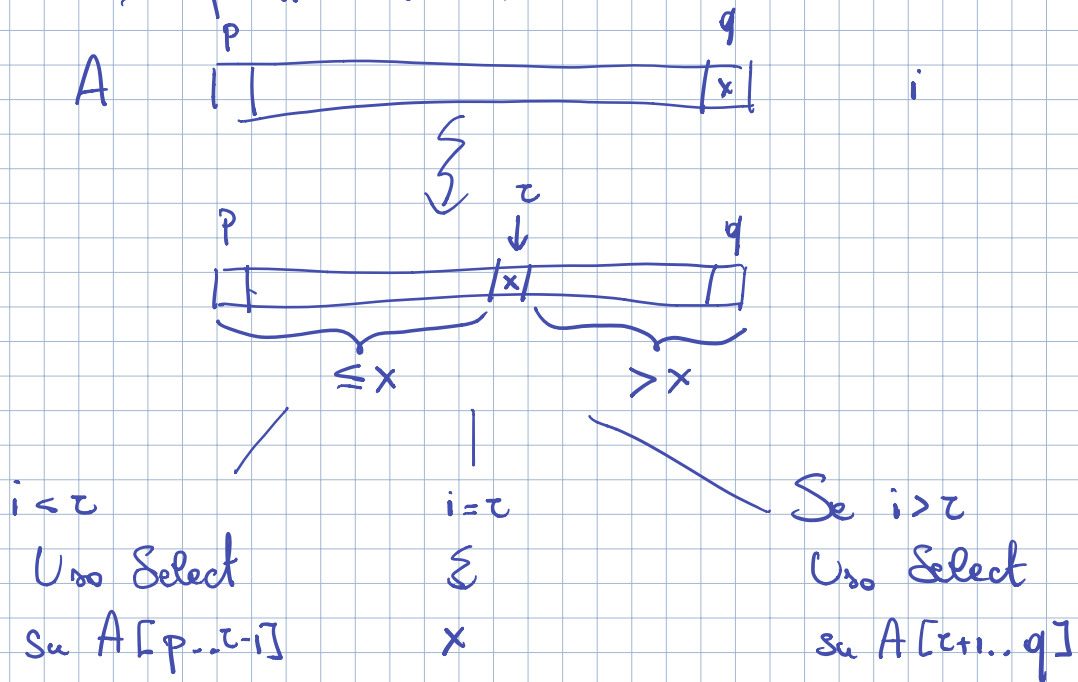
Ordino + Restituisco $A[i]$

Soluzione $O(n \log n)$

Selection

Idea

- Quick Partition



SelectNaire (A, p, q, i) { $\% p \leq i \leq q$

if ($p = q$) {
 return $A[p]$

} else {

→ $z \leftarrow \text{Partition}(A, p, q)$

if ($i = z$) {
 return $A[z]$

}

```

    if (i < r) {
        return SelectNaive(A, p, r-1, i)
    } else {
        return SelectNaive(A, r+1, q, i)
    }
}

```

$$T(n) = \begin{cases} \textcircled{n}(1) & n \leq 1 \\ T(m) + \textcircled{n}(m) & n > 1 \end{cases}$$

$$0 \leq m \leq n-1$$

Case fortunate $m=0 \rightsquigarrow \textcircled{n}(n)$

Case $m = \frac{n}{2} \rightsquigarrow \textcircled{n}(n)$

Case $m = n-1$

$$\begin{array}{l} \bullet \quad c \cdot n \\ | \\ \bullet \quad c(n-1) \\ \vdots \\ \bullet \quad \vdots \end{array} \rightsquigarrow \textcircled{n}(n^2)$$

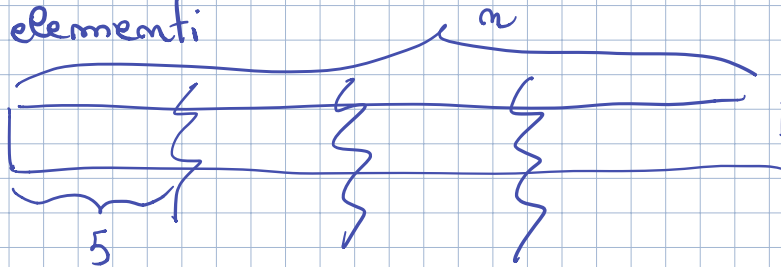
- Dato assicurare che sia a sx che a dx del primo finiscano abbastanza elementi



Un ricorrenza
Select anche per
la scelta del primo
Un select su un sottovettore

Select

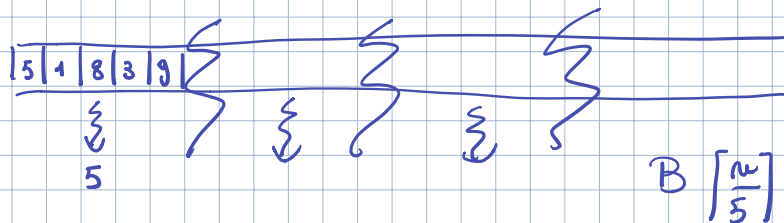
- Divido il vettore in blocchetti da 5 elementi



$\lceil \frac{n}{5} \rceil$ blocchetti

- Cerco il mediano di ogni blocchetto e lo copio in un vettore B

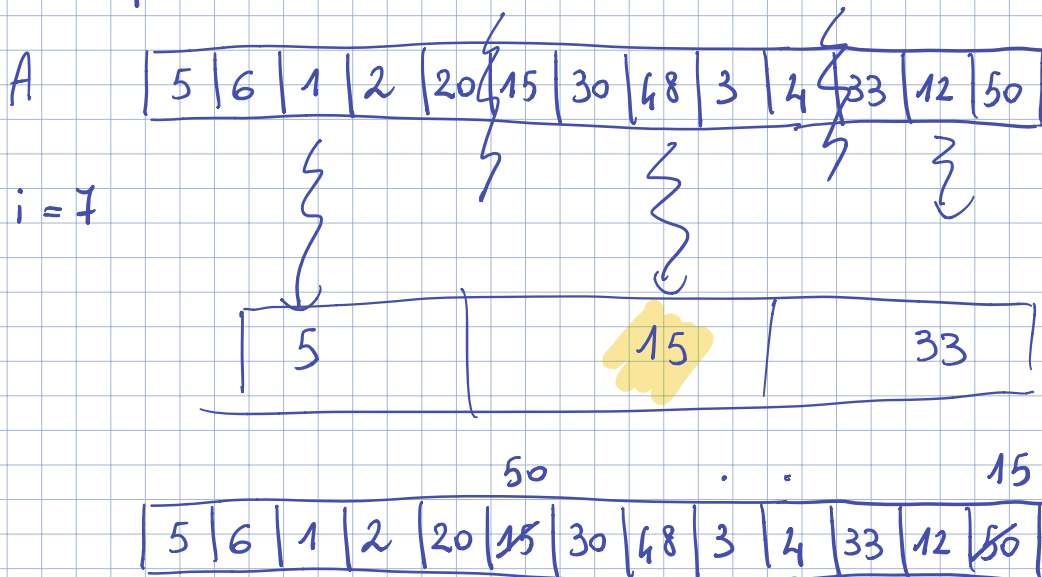
$O(n)$

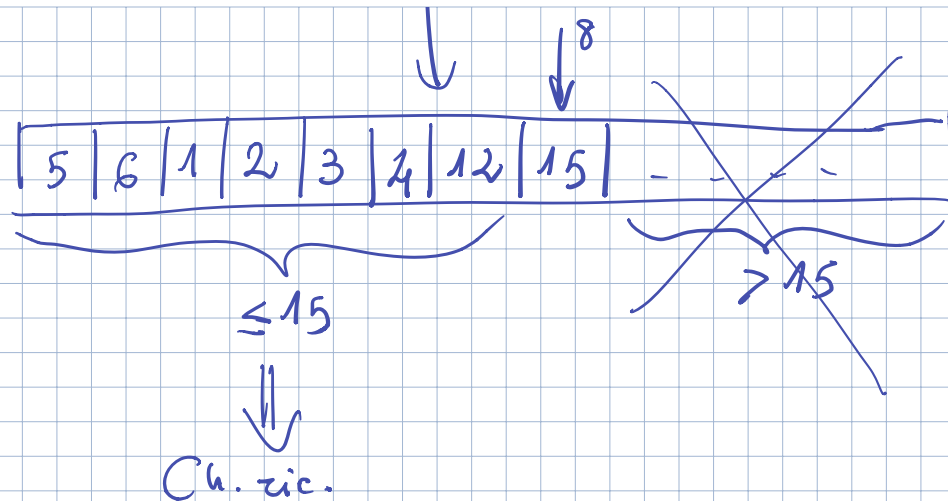


• Cerco in B l'elemento mediano
 $T\left(\frac{n}{5}\right)$ M
 M finirebbe in posizione $\left\lceil \frac{n}{5} \right\rceil \cdot \frac{1}{2}$ e
 ordinare B $\leadsto$ Usa Select (B, $\left\lceil \frac{n}{5} \right\rceil \cdot \frac{1}{2}$)

• Usa M come perno per il vettore
 A
 (4)(n) $\leadsto$ - Richiamo Partition con perno M
 $T(n)$ $\leadsto$ - Chiama ric. su A a sx
 o a dx di M

Esempio





Complessità

$$T(n) = \begin{cases} \Theta(1) & n \leq 1 \\ T\left(\frac{n}{5}\right) + T(n) + \Theta(n) & n > 1 \end{cases}$$

Annotations for the recurrence relation:

- $\alpha \cdot n$ (under $T\left(\frac{n}{5}\right)$)
- $\beta \cdot n$ (under $T(n)$)
- $\alpha = \frac{1}{5}$
- $\beta \leq \frac{4}{5}$
- quanto vale n ? al massimo?

Obs.

$$T(n) = \begin{cases} \cancel{\Theta(1)}^2 & n \leq 1 \\ T(\alpha \cdot n) + T(\beta \cdot n) + \cancel{\Theta(n)}^{b \cdot n} & n > 1 \end{cases}$$

Annotations for the recurrence relation:

- $\alpha + \beta < 1$
- allora $T(n) = \Theta(n)$
- $1 - (\alpha + \beta) > 0$
- $\frac{n}{1 - (\alpha + \beta)}$ el. fuori dalle ch. ric.

1. Dimostrare che Oss è vera

2. Determinare m (β) nel caso di Select.

1. Dim Oss.

$$\S) \exists c > 0 \exists \bar{n} \forall n \geq \bar{n} T(n) \leq c \cdot n$$

BASE $\bar{n} = 1$

$$T(1) = 2 \stackrel{?}{\leq} c \cdot 1 \quad \leadsto c \geq 2$$

PASSO

$$T(n) = T(\alpha \cdot n) + T(\beta \cdot n) + b \cdot n$$

$$\leq c \cdot (\alpha \cdot n) + c \cdot (\beta \cdot n) + b \cdot n$$

$$\stackrel{\text{Hplud}}{\leq} c(\alpha + \beta) \cdot n + b \cdot n \stackrel{??}{\leq} c \cdot n \quad \leftarrow$$

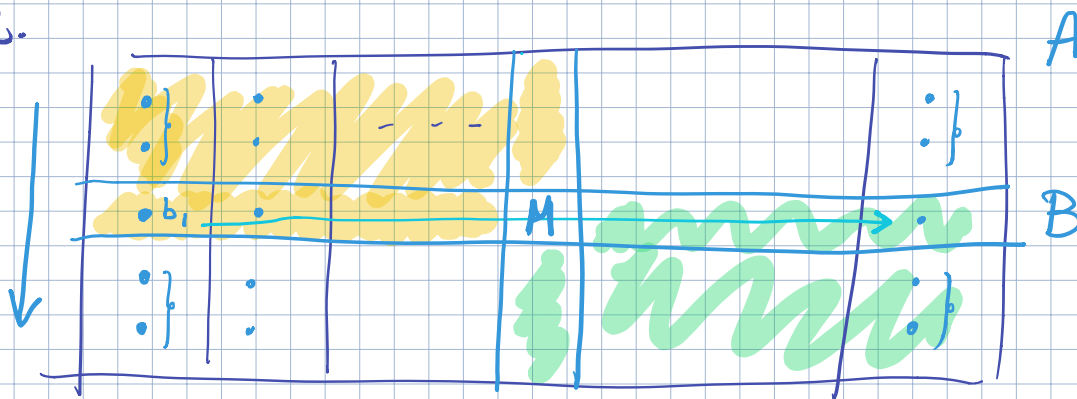
$$\rightarrow c \cdot \overbrace{(1 - (\alpha + \beta))}^{>0} \geq b \quad \downarrow \alpha + \beta < 1$$
$$c \geq \frac{b}{1 - (\alpha + \beta)}$$

$$c(\alpha + \beta) \cdot x + b \cdot x \leq c \cdot x$$

$$\underbrace{-c(\alpha + \beta)} + c(\alpha + \beta) + b \leq c$$

$$b \leq c - c(\alpha + \beta)$$

2.

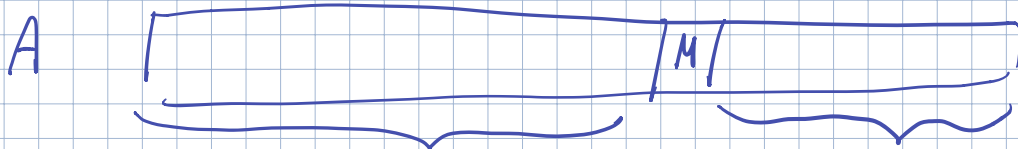


Quanti elementi di A sono sicuramente $< M$  $\leadsto \frac{1}{4} \cdot n$ elementi $< M$

• In A ci sono al più $\frac{3}{4} n$ elementi $> M$

$> M$

• In A ci sono al più $\frac{3}{4} n$ elementi $< M$



$$\Rightarrow \frac{3}{4} \cdot n$$

$$\leq \frac{3}{4} \cdot n$$

$$m \leq \frac{3}{4} n$$

$$T(n) \leq \begin{cases} \Theta(1) & n \leq 1 \\ T\left(\frac{n}{5}\right) + T\left(\frac{3}{4}n\right) + \Theta(n) & n > 1 \end{cases}$$

$$\frac{1}{5} + \frac{3}{4} < 1$$

$$\frac{1}{3} + \frac{3}{4} > 1$$

$$\frac{1}{7} + \frac{3}{4} < 1$$



$$T(n) = \Theta(n)$$

Select $\Theta(n)$
 Non è in-place
 Non è stabile

QuickSort-off $\Theta(n \log n)$

QuickSort-ott (A, p, q) {

if (p < q) {

⊕(n) { x ← Select (A, p, q, $\frac{q-p+1}{2}$)

{ z ← Partition (A, p, q, x)

2 T($\frac{n}{2}$) { QuickSort-ott (A, p, z-1)

{ QuickSort-ott (A, z+1, q)

}

⊕(n log n)

~ . ~ .

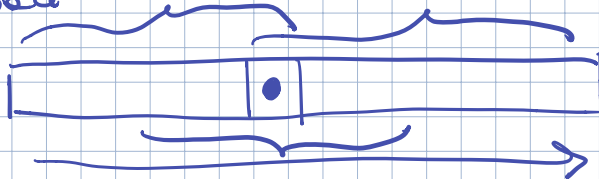
Esercizio

Dato **A** vettore determinare se esiste
x che compare in A più di $\frac{n}{2}$ volte

⊕(n)

Sfruttando Select?

n dispari



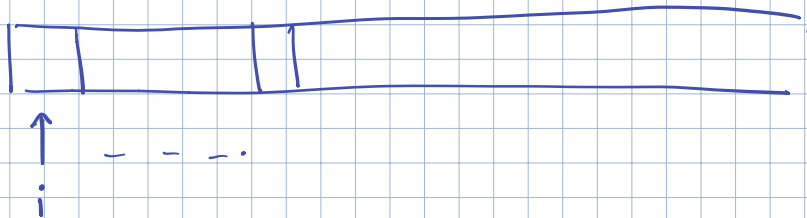
Richiamo $\text{Select}(A, \lceil \frac{n}{2} \rceil) \leadsto x$

- Controlla quante volte compare x in A

Non è in-place

Soluzione in-place e $\Theta(n)$?

A non è ordinato



$\text{count} \leftarrow 1$

$\text{candidate} \leftarrow A[1]$

Se incontro il candidate $\leadsto$ incremento

Se incontro $x \neq \text{candidate} \leadsto$ decremento

Se il contatore è zero $\leadsto$ aggiornare il
candidate

Majority Candidate (A) {

count $\leftarrow$ 0

for (i $\leftarrow$ 1 to A.length) {

if (count = 0) {

candidate $\leftarrow$ A[i]

} else {

if (A[i] = candidate) {

count $\leftarrow$ count + 1

} else {

count $\leftarrow$ count - 1

}

}

}

return candidate

}

② (n)

in-place

✓

